BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010 MLOPs Report

Group ID:25-26J-089

Project Title: Data-Driven Analysis of University Past Papers and Lecture Content for Curriculum Optimization

1. IT Number: IT22891440, Individual Component: AI supported development of model exam paper and aligned short notes
2. IT Number: IT22171306, Individual Component: NLP-Based Study Material Generation and Continuous Assessment Guidance Tool
3. IT Number: IT22598196, Individual Component: An Intelligent Essay-Based Learning Support System with Difficulty Prediction and Feedback-Driven Improvement
4. IT Number: IT22113672, Individual Component: An NLP-Based Exam-Oriented Study Plan and MCQ Recommendation System

		IT22891440	IT22171306	IT22598196	IT22113672
Data Pipeline	**Data Sources:**	• Historical DMS past exam papers collected from SLIIT second year IT undergraduate assessments, uploaded as PDF files including both digitally generated and scanned documents • DMS lecture slide materials uploaded as PDF files for semantic	• DMS module lecture PDF files provided by SLIIT serve as the primary knowledge base for all four core features • Student-submitted assignment and lab sheet documents in PDF format are accepted as input for the CA	• DMS past exam papers collected from SLIIT second year IT undergraduate assessments, uploaded as PDF files including both digitally generated and scanned documents • DMS lecture materials uploaded as PDF files used as the	• Historical DMS past examination MCQ papers collected from second year IT undergraduate assessments at Sri Lanka Institute of Information Technology • DMS lecture slide materials provided in PDF format, used as the primary source for topic

		indexing and retrieval • MongoDB Atlas used for storing extracted templates, canonical templates, and generated papers under the pastpaper_db database with three collections: canonical_templates, templates, and papers.	Guidance feature • Optional datasets uploaded by students in SQL or CSV format are used for query-based guidance with table outputs	primary knowledge base for question extraction and topic identification • A labeled dataset (`final_bloom_dataset.csv`) containing Bloom taxonomy categories used for training the difficulty classification model • Student-submitted essay answers collected through the system interface for evaluation and feedback generation • System generated attempt logs stored in CSV	coverage and concept reference • Student quiz responses and correct answer keys • Derived quiz outputs Retrieved the recommendation context. • Derived quiz outputs Retrieved recommendation context.

				format (`attempt_log.csv`) for tracking student performance and analytics.	
	Data Preprocessing:	• Past papers processed through a multi-stage preprocessing pipeline using the following scripts: pastpaper_extract ,lectureslide_extract,structure_topics_template, template_analyzer, and optionally migrate_data for MongoDB Atlas sync • PDF text extraction performed using pdfplumber and PyMuPDF for digitally generated PDFs	• Lecture PDF files are processed using PyMuPDF to extract machine-readable text from each page using reading-order extraction with a block-level fallback for scanned or image-heavy slides • Images embedded within the lecture PDFs are extracted and saved to the local file system for retrieval	• PDF text extraction performed using PyMuPDF for digitally generated documents • Extracted text cleaned by removing unnecessary symbols, formatting issues, and noise • Valid essay questions filtered using rule-based validation techniques • Questions classified into difficulty levels	• PDF text extraction performed using PyMuPDF fo r digitally generated document • Parsed MCQs into structured fields (question, options, answer) • Mapped each MCQ to topic/concept using lightweight metadata/similarity logic. • Cleaned and normalized text for retrieval and summary generation.

		• OCR extraction performed using pytesseract and OpenCV-python-headless for scanned documents • Topic clustering applied to identify subject areas across historical papers • Blueprint analysis performed to extract question formats and mark distributions across multiple years of assessments • Template mining applied to identify structural patterns stored as canonical and raw templates • Lecture slides chunked into semantically meaningful	during summarization • Extracted text is converted into LangChain Document objects and split into chunks using `RecursiveCharacterTextSplitter` with a chunk size of 500 characters and an overlap of 50 characters • Both text and image embeddings are generated using the `CombinedEmbeddings` model to support multimodal retrieval • Resulting embedding vectors are normalized using	using a Bloom-based classification model • Topic detection performed using an LLM-based approach for identifying key subject areas • Student answers normalized and processed before evaluation • Batch answer evaluation implemented where multiple answers are evaluated together, requiring a minimum threshold of correct answers to pass	• Built weak-topic lecture chunk retrieval pipeline for RAG summaries

		segments and embedded using sentence-transformers/all-MiniLM-L6-v2 producing 384-dimensional vectors • Vectors L2 normalized before indexing into a FAISS flat inner-product index to enable similarity-based retrieval.	L2 normalization before being stored in the FAISS index to enable cosine similarity-based retrieval		
	Data storage and versioning	• Extracted canonical templates and raw templates stored in MongoDB Atlas under pastpaper_db • Generated model papers stored in MongoDB Atlas papers collection	• FAISS is used as the vector store for semantic retrieval, with the index persisted to disk and reloaded on subsequent runs to avoid unnecessary recomputation • MongoDB is used as the	• Extracted questions stored in an in-memory Question Bank with attributes such as question text, difficulty level, topic, source, and page number • Trained Bloom classification model stored as	• Runtime state is managed in memory during execution to enable fast processing of quiz evaluation, adaptive study plan generation, and mastery estimation • Lecture-wise performance

		• FAISS index persisted to disk as slides_faiss_index _flatip. index and reloaded on subsequent runs to avoid recomputation • GitHub used for code versioning in a monorepo structure containing all four backend components and one shared frontend	primary document store for all structured application data including user profiles, base and reinforced summaries, flashcard sets, feedback records, and CA guidance reports • Audio files generated by the Piper TTS engine are stored on the local file system under a dedicated outputs directory • Summary and guidance versioning is managed within MongoDB through version fields and summary type identifiers,	`bloom_model.pkl` for reuse in prediction • Reinforcement learning policy stored as `rl.pkl` for adaptive learning decisions • Student attempt history stored in CSV format for analytics and tracking • GitHub used for code versioning and maintaining backend system updates	data, quiz responses, and learning sequences are temporarily stored for real-time computation • Sequence data is utilized for probabilistic student modeling using Bayesian Knowledge Tracing • Generated outputs are persistently stored in structured formats such as JSON, CSV, HTML, and Markdown for analysis and reproducibility • Stored artifacts include adaptive study plans, lecture importance scores, accuracy

			distinguishing between base outputs and reinforced outputs generated from student feedback		distributions, and BKT-based mastery estimates • Code versioned with Git
Model Development	**Model Selection**	• GPT-4o-mini selected as the LLM for question generation, quality critique, and short notes synthesis, accessed via the OpenAI API through a configurable LLM factory module • sentence-transformers/all-MiniLM-L6-v2 selected as the embedding model for semantic retrieval over lecture slide chunks	• GPT-4o was selected for CA Guidance generation due to its strong reasoning capability and capacity for handling long, complex assignment documents with detailed multi-part outputs • GPT-4o-mini was selected for topic summarization and flashcard generation, offering a balance between	• LinearSVC with TF-IDF selected as the primary model for Bloom-based difficulty classification due to its high accuracy • OpenAI GPT-based models used for answer evaluation, feedback generation, and topic detection • Reinforcement Learning (Q-learning approach) used for adaptive	• Hybrid retrieval approach: lexical + embedding + graph-based linkage • Sentence embeddings for semantic matching (all-MiniLM-L6-v2) • GraphRAG-style weak-topic -> related-topic -> MCQ recommendation flow • Bayesian Knowledge Tracing (BKT) added as an additional

		• FAISS flat inner-product index selected for efficient similarity-based vector retrieval • Model selection was based on GPT-4o-mini's balance between generation quality and response efficiency within the 300 second gateway timeout constraint	output quality and efficiency for structured educational content generation • FAISS with a RAG pipeline was selected for knowledge retrieval to ensure that generated content remained grounded in the DMS lecture materials • Piper TTS was selected for audio generation to provide voice notes alongside summaries, supporting auditory learners within the student user group	difficulty progression • n8n webhook integration used for generating additional practice questions when student performance is below threshold	mastery signal in adaptive planning • LLM-based summary generation

	Model Training	• No model fine tuning performed as the system relies entirely on pre-trained foundation models accessed via the OpenAI API • Prompt engineering techniques applied to GPT-4o-mini through structured prompts incorporating historical templates, topic trends, retrieved lecture context, and slot-specific constraints • The FAISS index was built from DMS lecture slide PDFs using the all-MiniLM-L6-v2 embedding	• No custom model training was performed in this system as all language models used are pre-trained foundation models accessed via the OpenAI API • The RAG pipeline was configured to ground model outputs in the DMS lecture content by retrieving the most semantically relevant chunks from the FAISS vectorstore before passing them as context to the language model • The FAISS index was built from	• Dataset final_bloom_dataset.csv used for training the classification model • TF-IDF vectorization applied with n-gram features to represent textual data • Multiple models evaluated including LinearSVC, Logistic Regression, and Multinomial Naive Bayes • LinearSVC selected as the final model based on performance metrics • Bloom taxonomy levels mapped into difficulty	• No custom end-to-end training pipeline for core runtime models • Used pretrained embedding model (all-MiniLM-L6-v2) • Applied deterministic scoring, threshold rules, and adaptive formulas on quiz-derived evidence • For offline research evaluation, retrieval methods were benchmarked using IR metrics (P@10, R@10, MRR, NDCG@10, Diversity@10)

		model, serving as the primary knowledge customization step allowing the system to produce DMS-specific outputs without fine tuning • The Question Writer agent uses carefully designed prompts enriched with past paper patterns and lecture content to guide GPT-4o-mini in generating curriculum aligned exam questions • The Quality Critic agent uses LLM-based review prompts to validate generated questions against	the DMS lecture PDFs using the CombinedEmbeddings model, which encodes both text and image content into a unified embedding space • This ingestion process serves as the primary knowledge customization step, allowing the system to produce DMS-specific outputs without requiring fine-tuning of the underlying language models	categories (Easy, Medium, Hard) • Trained model serialized and saved for integration into the backend system	

		academic quality standards			
Model Integration	**Tools used**	• Backend: Python + FastAPI, Uvicorn • Frontend: React 19 + Vite 7 • Database: MongoDB Atlas • IDE: VS Code • API Gateway: Nginx • LLM: GPT-4o-mini via OpenAI API • Embedding Model: sentence-transformers/all-MiniLM-L6-v2 • Vector Store: FAISS • Additional libraries: pdfplumber, PyMuPDF, pytesseract, opencv-python-headless, fpdf2,	• FastAPI was used as the backend web framework to expose all model-driven features as REST API endpoints • CrewAI was used to implement the multi-agent architecture, coordinating agents responsible for summarization, guidance generation, flashcard creation, reinforcement, and scheduling • LangChain was used to build the RAG pipeline,	• Backend: Python + FastAPI, Uvicorn • Machine Learning: scikit-learn, pandas, numpy • PDF Processing: PyMuPDF • LLM Integration: OpenAI API • Workflow Automation: n8n • API Testing: Postman • Environment Configuration: python-dotenv	• Backend: Python + FastAPI • Frontend: React • Data/ML utilities: Pandas, NumPy, scikit-learn • Embeddings: SentenceTransformers (all-MiniLM-L6-v2) • Retrieval/indexing: similarity-based ranking pipeline • PDF processing: pdfplumber, PyPDF2, PyMuPDF (as used in your pipeline) • Graph visualization: PyVis • LLM APIs: Gemini for higher-quality

		motor, pymongo, scikit-learn, numpy, pandas	managing the vectorstore retrieval chain and document processing utilities • OpenAI Python SDK was used to interface with GPT-4o and GPT-4o-mini for content generation tasks • PyMuPDF was used for PDF text and image extraction during both lecture ingestion and assignment processing • Authlib was used for Google OAuth 2.0 integration and the Google Calendar API was used for automated		weak-topic summaries

			deadline event creation • Piper TTS was integrated as a local text-to-speech engine for audio summary generation • MongoDB with PyMongo was used for structured data persistence • FAISS with LangChain Community was used for vector storage and semantic retrieval		
Model Deployment	**Testing Environments**	• All development and testing activities were conducted on a local machine environment • API endpoints were tested using	• All development and testing activities were conducted on a local machine environment • API endpoints were tested	• All development and testing conducted on a local machine environment • API endpoints tested using Postman on the	• Local machine development/testing environment • API endpoint testing for quiz, adaptive plan, graph, and weak-

		Postman on the local development server • The full system was verified through manual browser-based testing with the backend running on Uvicorn as the ASGI server	using Postman on the local development server • The full system was verified through manual browser-based testing with the backend running on Uvicorn as the ASGI server	local development server • System functionality validated through browser-based testing with FastAPI running on Uvicorn • Batch evaluation system tested for both pass and fail scenarios	topic summary flows • Browser-based UI verification for full student workflow • Offline research validation scripts for retrieval comparison and fairness checks
	Deployment Platform	• The system was deployed on Microsoft Azure using a Virtual Machine instance • The Azure VM provided a stable and accessible environment for external users including the second year IT students at SLIIT who participated	• The system was deployed on Microsoft Azure using a Virtual Machine instance • The Azure VM provided a stable and accessible environment for external users including the DMS students	• The system is designed to be deployed on Microsoft Azure Virtual Machine • Backend services are hosted using FastAPI and accessible through public endpoints	• The system was deployed on Microsoft Azure utilizing a Virtual Machine (VM) instance to ensure scalability and reliability.

		in the user acceptance testing phase	who participated in the user acceptance testing phase		
	Deployment Method	• The required Python environment was set up on the Azure VM and all dependencies were installed from requirements.txt • Environment variables including the OpenAI API key, MongoDB Atlas URI, and other configuration settings were configured on the VM • The FastAPI application was run using Uvicorn as the ASGI server on the Azure VM • The frontend was configured to	• The required Python environment was set up on the Azure VM and all dependencies were installed • Environment variables including API keys, MongoDB URI, and Google OAuth credentials were configured on the VM • The FastAPI application was run using Uvicorn as the ASGI server on the Azure VM • The frontend was configured to communicate	• Python environment configured with required dependencies using `requirements.txt` • Environment variables such as API keys and webhook URLs configured using. env • FastAPI application deployed using Uvicorn as the ASGI server • Question bank initialized during system startup • Frontend configured to communicate with backend	• The Azure VM provided a stable and remotely accessible environment, enabling external users—including second-year IT students at Sri Lanka Institute of Information Technology—to participate effectively in the User Acceptance Testing (UAT) phase. • The backend application, developed using FastAPI, was deployed and executed via Uvicorn, ensuring high-performance

		communicate with the deployed backend via the VM's public IP address • The Nginx API gateway was configured to route requests to the model paper generation service, enforcing the 100MB upload limit and 300 second request timeout	with the deployed backend via the VM's public IP address • Static file serving for images and audio was handled directly through the FastAPI application using mounted static directories	through REST APIs • CORS settings configured to allow secure communication between frontend and backend	asynchronous request handling. • The frontend interface was integrated with the backend services through the VM's public IP address, facilitating seamless client-server communication.
Future Enhancements	**Model Improvement:**	• Expand the system beyond the SLIIT DMS module to support model paper generation and aligned short notes for other university subjects and modules, making the	• The RAG pipeline could be enhanced by incorporating fine-tuned embedding models trained specifically on DMS content to improve retrieval precision for highly technical concepts	• Improve classification accuracy using larger and more diverse datasets • Expand the system to support additional university modules beyond DMS • Enhance answer evaluation by	• Improve topic-label accuracy with manual gold-labeled subset validation • Perform sensitivity testing for hybrid weighting strategies • Expand BKT from lecture-level to topic-level

		framework reusable across different academic departments • Introduce multi-user access management and institutional account support, allowing any university or academic institution to adopt the platform by uploading their own module-specific past papers and lecture slides, enabling the system to serve multiple institutions, departments,	• The CA Guidance feature could be extended to support additional file formats beyond PDF and CSV, including Excel and Word documents • The ER Diagram Generator could be improved with an interactive drag-and-drop canvas allowing students to refine generated diagrams directly within the interface • The Piper TTS audio generation could be upgraded to support multiple voices and languages, making the	combining LLM-based and rule-based scoring techniques • Implement database storage for long-term tracking of student learning progress • Improve personalization by analyzing individual student learning behavior over time • Enhance n8n workflow to generate more context-aware and adaptive practice questions • Integrate automated notification system using n8n to send alerts and feedback to	sequential mastery tracking • Add confidence scoring and uncertainty indicators to recommendations • Enhance weak-topic summary grounding checks to reduce potential paraphrase drift • Add student-level longitudinal adaptation for multi-attempt learning trajectories • Develop a mobile application version to allow students to access study plan generation and recommended MCQ questions.

		and student cohorts simultaneously while keeping each institution's data separate and private • Develop a mobile application version to allow students to access generated model papers and short notes on the go	system more accessible to a diverse student population • The platform could be adapted for use by other universities and institutions by replacing the DMS lecture content with module-specific materials from any subject area, extending the same features to students across different disciplines and curricula • A mobile application version of the system could be developed to allow students to access summaries,	students via email and WhatsApp for improved engagement and real-time learning support • Develop a mobile application version with email and WhatsApp notification support through n8n, allowing students to receive feedback alerts, practice reminders, and progress updates on the go.	

			flashcards, and guidance on the go • The feedback and reinforcement mechanism could be further enhanced with a more sophisticated personalization engine that tracks individual student learning patterns over time and proactively recommends topics for review based on their engagement history		